41039 Programming 1

Revision of Week 12: Review and integration



Dr. Xuan-Bach Le
le.xuan.bach@hcmut.edu.vn

Faculty of Computer Science and Engineering, HCMUT

September Session 2026

How the revision session works

A recap before the next lecture

Write an answer before the reveal. Explain the state change, rule or contract that makes it follow. These are practice questions, with no new assessment requirements.

Part 1: on your own

- 8 short questions
- About 90 seconds per question
- Java and Python; name the language first

Part 2: teams of 3 or 4

- Draw the model
- Spot the errors
- Predict the output
- Write and test the code

A result needs a reason. A repair needs a counterexample.

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

1. A loop with an explicit invariant

Java 8 excerpt: main statements and supporting types; commit to an outcome before the reveal.

```
int[] marks={49,50,85};
int passed=0;
for(int i=0;i<marks.length;i++) {
    if(marks[i]>=50) passed++;
}
System.out.println(passed);
```

Your answer

What is the complete output? Which line or rule decides it?

1. A loop with an explicit invariant

Java 8 excerpt: main statements and supporting types; commit to an outcome before the reveal.

```
int[] marks={49,50,85};
int passed=0;
for(int i=0;i<marks.length;i++) {
    if(marks[i]>=50) passed++;
}
System.out.println(passed);
```

Answer

2

Exactly 50 is included, so the marks 50 and 85 pass.

2. Mutation versus replacement, revisited

Python 3; commit to an outcome before the reveal.

```
def change(values):
    values.append(3)
    values=[9]
    return values
items=[1,2]
result=change(items)
print(items)
print(result)
```

Your answer

What is the complete output? Which line or rule decides it?

2. Mutation versus replacement, revisited

Python 3; commit to an outcome before the reveal.

```
def change(values):
    values.append(3)
    values=[9]
    return values
items=[1,2]
result=change(items)
print(items)
print(result)
```

Answer

```
[1, 2, 3]
[9]
```

Appending affects the shared list; assigning a new list to the local name does not rebind items.

3. An end-to-end marks pipeline

Java 8 excerpt: main statements and supporting types; commit to an outcome before the reveal.

```
import java.io.*;
String data="Linh,85\nAn,49\nMai,50";
int count=0,passed=0;
try(BufferedReader reader=new BufferedReader(new
     StringReader(data))) {
    String line;
    while((line=reader.readLine())!=null) {
        String[] parts=line.split(",");
        int mark=Integer.parseInt(parts[1]);
        count++;
        if(mark>=50) passed++;
    }
} catch(IOException e) { throw new RuntimeException(e);
     }
System.out.println("records="+count+", passed="+passed);
```

Your answer

What is the complete output? Which line or rule decides it?

3. An end-to-end marks pipeline

Java 8 excerpt: main statements and supporting types; commit to an outcome before the reveal.

```
import java.io.*;
String data="Linh,85\nAn,49\nMai,50";
int count=0,passed=0;
try(BufferedReader reader=new BufferedReader(new
     StringReader(data))) {
    String line;
    while((line=reader.readLine())!=null) {
        String[] parts=line.split(",");
        int mark=Integer.parseInt(parts[1]);
        count++;
        if(mark>=50) passed++;
    }
} catch(IOException e) { throw new RuntimeException(e);
     }
System.out.println("records="+count+", passed="+passed);
```

Answer

```
records=3, passed=2
```

All three valid simple records are read, including the final line without a newline; two marks pass.

4. Separate parse, validate and summarise

Python 3; commit to an outcome before the reveal.

```
def parse(line):
    name,text=line.split(",")
    mark=int(text)
    if not 0<=mark<=100:
        raise ValueError("range")
    return name,mark
records=[]
for line in ["Linh,85","An,bad","Mai,50"]:
    try: records.append(parse(line))
    except ValueError: print("Invalid")
print(sum(mark>=50 for name,mark in records))
```

Your answer

What is the complete output? Which line or rule decides it?

4. Separate parse, validate and summarise

Python 3; commit to an outcome before the reveal.

```
def parse(line):
    name,text=line.split(",")
    mark=int(text)
    if not 0<=mark<=100:
        raise ValueError("range")
    return name,mark
records=[]
for line in ["Linh,85","An,bad","Mai,50"]:
    try: records.append(parse(line))
    except ValueError: print("Invalid")
print(sum(mark>=50 for name,mark in records))
```

Answer

```
Invalid
2
```

The bad record is not appended; Linh and Mai both pass.

5. A contract can be tested at its boundaries

Java 8 excerpt: main statements and supporting types; commit to an outcome before the reveal.

```
Counter counter=new Counter(2);
System.out.println(counter.take(0));
System.out.println(counter.take(2));
System.out.println(counter.take(1));
System.out.println(counter.left());
class Counter {
    private int count;
    Counter(int count) { this.count=count; }
    boolean take(int n) {
        if(n<=0 || n>count) return false;
        count-=n;
        return true;
    }
    int left() { return count; }
}
```

Your answer

What is the complete output? Which line or rule decides it?

5. A contract can be tested at its boundaries

Java 8 excerpt: main statements and supporting types; commit to an outcome before the reveal.

```
Counter counter=new Counter(2);
System.out.println(counter.take(0));
System.out.println(counter.take(2));
System.out.println(counter.take(1));
System.out.println(counter.left());
class Counter {
    private int count;
    Counter(int count) { this.count=count; }
    boolean take(int n) {
        if(n<=0 || n>count) return false;
        count-=n;
        return true;
    }
    int left() { return count; }
}
```

Answer

```
false
true
false
0
```

Zero is rejected; taking exactly 2 succeeds; taking 1 from zero stock is rejected.

6. Make a useful explanation from a trace

Python 3; commit to an outcome before the reveal.

```
marks=[49,50,85]
total=0
for position,mark in enumerate(marks):
    before=total
    total+=mark
    print(position,before,total)
print("total",total)
```

Your answer

What is the complete output? Which line or rule decides it?

6. Make a useful explanation from a trace

Python 3; commit to an outcome before the reveal.

```
marks=[49,50,85]
total=0
for position,mark in enumerate(marks):
    before=total
    total+=mark
    print(position,before,total)
print("total",total)
```

Answer

```
0 0 49
1 49 99
2 99 184
total 184
```

The accumulator moves $0 \to 49 \to 99 \to 184$; the trace connects each new state to its cause.

7. Optional extension: filter, map, reduce

Java 8 excerpt: main statements and supporting types; optional extension; commit to an outcome before the reveal.

```
import java.util.*;
List<Integer> marks=Arrays.asList(49,50,85);
int total=marks.stream()
    .filter(mark -> mark>=50)
    .mapToInt(mark -> mark)
    .sum();
System.out.println(total);
System.out.println(marks);
```

Your answer

What is the complete output? Which line or rule decides it?

7. Optional extension: filter, map, reduce

Java 8 excerpt: main statements and supporting types; optional extension; commit to an outcome before the reveal.

```
import java.util.*;
List<Integer> marks=Arrays.asList(49,50,85);
int total=marks.stream()
    .filter(mark -> mark>=50)
    .mapToInt(mark -> mark)
    .sum();
System.out.println(total);
System.out.println(marks);
```

Answer

```
135
[49, 50, 85]
```

The pipeline sums 50 and 85 without changing the source list. This uses Java 8-compatible APIs.

8. Optional extension: a pure transformation

Python 3; optional extension; commit to an outcome before the reveal.

```
def doubled(values):
    return [value*2 for value in values]
source=[2,3]
print(doubled(source))
print(source)
print(doubled(source)==doubled(source))
```

Your answer

What is the complete output? Which line or rule decides it?

8. Optional extension: a pure transformation

Python 3; optional extension; commit to an outcome before the reveal.

```
def doubled(values):
    return [value*2 for value in values]
source=[2,3]
print(doubled(source))
print(source)
print(doubled(source)==doubled(source))
```

Answer

```
[4, 6]
[2, 3]
True
```

For the same integer input this function returns the same result and does not modify its argument.

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

Form teams

- Teams of 3 or 4. One shared sheet, one spokesperson.
- For each challenge, record the result and the reason.
- Rotate the spokesperson. Other teams can challenge an explanation.
- The final task has starter files and a Java/Python solution pair.

Discuss first. Commit to an answer. Then compare with the reveal.

Draw it: The valid-record pipeline

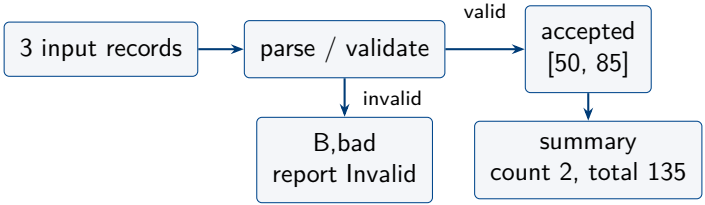
```
import java.util.*;
String[] lines={"A,50","B,bad","C,85"};
List<Integer> accepted=new ArrayList<Integer>();
for(String line:lines) {
    try { accepted.add(Integer.parseInt(line.split(",")
     [1])); }
    catch(NumberFormatException e) { System.out.println(
     "Invalid"); }
}
int total=0; for(int mark:accepted) total+=mark;
System.out.println(accepted.size()+" "+total);
```

Course website: Week 12. Complete programs accompany this deck.

Four minutes

Draw the flow and responsibilities after the shown operations. Label the important state. Mark what is shared, what changes, or which path handles failure. Predict every output line.

Draw it: one answer



Output

```
Invalid
2 135
```

Attempted inputs and accepted records are different collections. Rejected data never contributes to a successful count or total. The shown numeric values are all within range.

Course website: Week 12. Complete programs accompany this deck.

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

Spot the errors 1: Java (5 min)

Contract

Reject non-numeric and out-of-range marks. Count and sum only accepted records.

```java
public class Broken {
    public static void main(String[] args) {
        String[] input={"A,50","B,bad","C,101"};
        int total=0,passed=0;
        for(String line:input) {
            try {
                int mark=Integer.parseInt(line.split(",")[1]);
                total+=mark;
                if(mark>50) passed++;
            } catch(NumberFormatException e) {
                System.out.println("Invalid");
            }
        }
        System.out.println("count,total,passed");
        System.out.println(input.length+","+total+","+passed);
    }
}
```

Spot the errors 1: answers

The repairs

- **L7:** After parsing, validate the domain range before updating any summary. Catch the validation failure too.
- **L9:** The pass boundary is inclusive: mark $>=$ 50.
- **L15:** Count accepted records, not attempted input records.

Course website: Week 12. Complete programs accompany this deck.

✗ Original program

```
Invalid
count,total,passed
3,151,1
```

✓ After the repairs

```
Invalid
Invalid
count,total,passed
1,50,1
```

Spot the errors 2: Python (5 min)

Contract

Reject malformed fields and invalid marks without stopping later records. Only valid marks contribute to the report.

```python
lines=["A,50","broken","C,101"]
marks=[]
for line in lines:
    parts=line.split(",")
    try:
        marks.append(int(parts[1]))
    except ValueError:
        print("Invalid")
print("count,total,passed")
print(f"{len(lines)},{sum(marks)},{sum(m>50 for m in marks)}")
```

Spot the errors 2: answers

The repairs

- **L6:** Check the number of fields and a non-empty name before indexing.
- **L6:** Parse and validate 0..100 before appending.
- **L10:** Count successful records with len(marks).
- **L10:** Include exactly 50 in the pass count.

✗ Original program

```
(no standard output)
IndexError: list index out of
    range
```

✓ After the repairs

```
Invalid
Invalid
count,total,passed
1,50,1
```

Course website: Week 12. Complete programs accompany this deck.

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

Predict 1: References, collections and a boundary

```
import java.util.*;
public class Demo {
    static int count(List<Integer> xs) { int n=0; for(
     int x:xs) if(x>=50) n++; return n; }
    public static void main(String[] args) {
        List<Integer> a=new ArrayList<Integer>(Arrays.
     asList(49,50,85));
        List<Integer> b=a;
        List<Integer> c=new ArrayList<Integer>(a);
        b.remove(Integer.valueOf(49));
        c.set(0,0);
        System.out.println(a);
        System.out.println(c);
        System.out.println(count(a));
    }
}
```

Three minutes

Write every line in order. Explain the first state change, alias, dispatch decision or exception that controls the result. Then propose one change that alters the output.

Course website: Week 12. Complete programs accompany this deck.

Predict 1: answer

```
[50, 85]
[0, 50, 85]
2
```

Why

Alias mutation affects a. The copied container c changes independently. count tests the inclusive boundary on the current a.

Course website: Week 12. Complete programs accompany this deck.

Predict 2: A record either enters the model or does not

```
def parse(text):
    name,raw=text.split(",")
    mark=int(raw)
    if not 0<=mark<=100: raise ValueError("range")
    return name,mark
records=[]
for line in ["A,50","B,bad","C,85"]:
    try: records.append(parse(line))
    except ValueError: print("Invalid")
print(len(records),sum(mark for name,mark in records))
print(sum(mark>=50 for name,mark in records))
```

Course website: Week 12. Complete programs accompany this deck.

Three minutes

Write every line in order. Explain the first state change, alias, dispatch decision or exception that controls the result. Then propose one change that alters the output.

Predict 2: answer

```
Invalid
2 135
2
```

Why

The failed conversion prevents append. The two accepted tuples supply both the valid-record count and numeric summary.

Course website: Week 12. Complete programs accompany this deck.

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

Code it: specification

A complete marks pipeline

Read n then n records name,mark. Require exactly two fields, a non-empty name, and an integer mark in 0..100. Print Invalid immediately for each rejected record. Store accepted Student objects. Finish with count,total,passed and the summary of valid records.

Example input

```
4
Linh,85
An,bad
Mai,50
Tea,101
```

Expected output

```
Invalid
Invalid
count,total,passed
2,135,2
```

Eight minutes. One Java team and one Python team at the keyboard. Other teams produce a boundary test and explain which wrong implementation it would expose.

Code it: Java solution (1/2)

```java
import java.util.*;
Scanner sc=new Scanner(System.in);
int n=Integer.parseInt(sc.nextLine());
List<Student> records=new ArrayList<Student>();
for(int i=0;i<n;i++) {
    String line=sc.nextLine();
    String[] parts=line.split(",",-1);
    try {
        if(parts.length!=2 || parts[0].isEmpty()) throw new IllegalArgumentException();
        int mark=Integer.parseInt(parts[1]);
        if(mark<0 || mark>100) throw new IllegalArgumentException();
        records.add(new Student(parts[0],mark));
    } catch(IllegalArgumentException e) { System.out.println("Invalid"); }
```

Course website: Week 12. Complete programs accompany this deck.

Code it: Java solution (2/2)

```
14 }
15 int total=0,passed=0;
16 for(Student student:records) { total+=student.mark(); if(student.passed()) passed++; }
17 System.out.println("count,total,passed");
18 System.out.println(records.size()+","+total+","+passed);
19 class Student {
20     private final String name;
21     private final int mark;
22     Student(String name,int mark) { this.name=name; this.mark=mark; }
23     int mark() { return mark; }
24     boolean passed() { return mark>=50; }
25 }
```

Course website: Week 12. Complete programs accompany this deck.

Code it: Python solution

```python
class Student:
    def __init__(self,name,mark): self.name,self.mark=name,mark
    def passed(self): return self.mark>=50
def parse(line):
    parts=line.split(",")
    if len(parts)!=2 or not parts[0]: raise ValueError("record")
    mark=int(parts[1])
    if not 0<=mark<=100: raise ValueError("range")
    return Student(parts[0],mark)
n=int(input())
records=[]
for _ in range(n):
    try: records.append(parse(input()))
    except ValueError: print("Invalid")
print("count,total,passed")
print(f"{len(records)},{sum(s.mark for s in records)},{sum(s.passed() for s in records)}")
```

Course website: Week 12. Complete programs accompany this deck.

Code it: the tests

A slash separates input lines; (blank) denotes an empty line.

Test 1
`4 / Linh,85 / An,bad / Mai,50 / Tea,101`
Output: `Invalid / Invalid / count,total,passed / 2,135,2`

Test 2
`0`
Output: `count,total,passed / 0,0,0`

Test 3
`3 / A,0 / B,50 / C,100`
Output: `count,total,passed / 3,150,2`

Test 4
`3 / missing / ,50 / a,b,c`
Output: `Invalid / Invalid / Invalid / count,total,passed / 0,0,0`

Explain why each test is present, not only whether it passes.

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

What you should now be able to explain

- Explain a program through state changes and call contracts.
- Combine validation, models, collections and report formatting.
- Choose tests that distinguish a plausible wrong implementation.
- Treat functional transformations and streams as optional follow-on material.

Return to the matching week page for runnable examples and further practice. Bring one unresolved prediction or counterexample to the next class.

Course website: Week 12. Complete programs accompany this deck.